Stock Price Trend Prediction

SWE-452

Software Development Life Cycle (SDLC) II

Nathan Kennedy, Batyr Rasulov, Alejandro Barranco-Leyte

February 18, 2026

1. Introduction

1.1 Purpose

This document lays out the finalized design and technical requirements for the Stock Price Trend Prediction system. It covers everything from data needs and ML model specs to cloud deployment and the user-facing dashboard. The goal is to give the team (and anyone reviewing this) a clear picture of what we're building, how it fits together, and what success looks like, so that we can move into implementation with minimal guesswork.

1.2 Audience

This document is written for the project team (Nathan, Batyr, and Alejandro), our course instructor, and anyone who needs to understand the technical direction of this project. It's meant to be practical, not a theoretical exercise, but a working blueprint we actually follow.

1.3 Scope

The system predicts whether a given stock's closing price will go up or down on the next trading day. We're not trying to predict exact prices, that's a losing game with how noisy financial markets are. Instead, we focus on directional trend classification using historical price data and technical indicators. The system pulls data from Yahoo Finance via the yfinance Python library, trains ML classification models, evaluates them against standard metrics, and serves predictions through a Streamlit-based dashboard. The whole thing runs on Google Cloud Platform for scalability and easy deployment.

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
ML	Machine Learning - algorithms that learn patterns from data
GCP	Google Cloud Platform - our cloud infrastructure provider
CI/CD	Continuous Integration / Continuous Deployment - automated build and release pipelines
yfinance	Open-source Python library for downloading market data from Yahoo Finance
OHLCV	Open, High, Low, Close, Volume - the standard daily price data fields

SMA	Simple Moving Average - average closing price over a rolling window
EMA	Exponential Moving Average - weighted moving average giving more weight to recent prices
RSI	Relative Strength Index - momentum indicator measuring speed/change of price movements
MACD	Moving Average Convergence Divergence - trend-following momentum indicator
API	Application Programming Interface
GCS	Google Cloud Storage - object storage on GCP
GCR	Google Container Registry - Docker image hosting on GCP

2. System Overview

2.1 High-Level Description

At a high level, this system does three things: it pulls historical stock data, runs it through trained ML models to predict next-day price direction, and shows the results on a simple web dashboard. The pipeline goes like this yfinance fetches OHLCV data for whatever ticker the user picks, we compute a set of technical indicators (moving averages, RSI, MACD, volatility measures), feed those features into classification models (Logistic Regression and Random Forest to start), and output a prediction: up or down for tomorrow's close.

The cloud side of things lives on GCP. We use Cloud Storage for raw and processed datasets, a Compute Engine instance (or Cloud Run, depending on traffic needs) for the Streamlit app, and Cloud Build for CI/CD so that every push to main triggers a fresh build and deploy. The ML training can run locally during development, but the production inference runs in the cloud.

2.2 System Architecture Diagram

The diagram below shows the main components and how data flows between them. Each layer is responsible for a specific job, and they communicate through well-defined interfaces.

Layer	Component	Technology	Responsibility

Data Ingestion	Data Fetcher	yfinance, Python	Pull OHLCV data from Yahoo Finance for selected tickers
Preprocessing	Feature Pipeline	Pandas, NumPy	Clean data, handle missing values, compute technical indicators (SMA, EMA, RSI, MACD, Bollinger Bands, daily returns, volatility)
Storage	Data Store	GCS Buckets, CSV/Parquet	Store raw data, processed features, and trained model artifacts
Model Training	Training Module	Scikit-learn, Joblib	Train Logistic Regression and Random Forest classifiers using time-series split
Evaluation	Eval Module	Scikit-learn metrics	Compute accuracy, precision, recall, F1 score, and confusion matrices
Serving	Prediction API	Streamlit / Flask	Load trained model, accept ticker input, return next-day trend prediction
Frontend	Dashboard	Streamlit	Display predictions, historical charts, model performance metrics
CI/CD	Build Pipeline	Cloud Build, GitHub	Auto-build and deploy on push to main branch

Data Flow Summary: Yahoo Finance → yfinance fetcher → raw CSV in GCS → preprocessing pipeline → feature CSV in GCS → model training → model artifact (.joblib) in GCS → Streamlit app loads model → user selects ticker → app fetches latest data → runs prediction → displays result on dashboard.

3. Functional Requirements

3.1 Data Input Requirements

The system needs to ingest historical stock price data reliably. Here's what that looks like in practice.

Data Source and Format

All market data comes from Yahoo Finance through the yfinance Python library (v0.2.x or later). Each API call returns a Pandas DataFrame with daily OHLCV data: that's Open, High,

Low, Close, and Volume indexed by date. We store this as CSV files for simplicity and portability, with Parquet as an option if we need to handle larger datasets down the road.

Pre-Processing Pipeline

The preprocessing step handles the messy reality of financial data:

- Drop rows where critical fields (Close, Volume) are missing. If less than 2% of data is missing, forward-fill. If more than that, flag the ticker for review.
- Normalize volume using min-max scaling so it's on a comparable scale with price-derived features.
- Compute the following technical indicators from raw OHLCV data: SMA (5-day, 10-day, 20-day), EMA (12-day, 26-day), RSI (14-day), MACD (12/26/9), Bollinger Bands (20-day, 2 std), daily returns (percent change), and 10-day rolling volatility.
- Generate the target label: 1 if tomorrow's close is higher than today's, 0 otherwise.
- Apply a time-series-aware train/test split 80% training, 20% test — with no shuffling, so we don't leak future data into training.

3.2 ML Model Requirements

Objective

The ML models classify each trading day as either "up" (price goes higher tomorrow) or "down" (price goes lower). This is a binary classification task. We're targeting 60-65% accuracy, which is a realistic and useful range for financial trend prediction: anything above random (50%) that holds up on out-of-sample data has value.

Models

We're starting with two well-understood models and may add a third depending on results:

- **Logistic Regression** - our baseline. Fast to train, easy to interpret, gives us a probability score we can threshold. Regularization via C parameter tuning.
- **Random Forest Classifier** - handles non-linear feature interactions well, less prone to overfitting with proper depth limits. We'll tune n_estimators (100-500), max_depth (5-20), and min_samples_split.
- **Gradient Boosted Trees (optional)** - if time allows, we'll try XGBoost or LightGBM for potentially better accuracy. These are more complex but often outperform on tabular data.

Training Details

Training uses scikit-learn's API. We use TimeSeriesSplit with 5 folds for cross-validation (not random K-fold, since that would break temporal ordering). Hyperparameter tuning is done

via GridSearchCV on the training set only. The final model is evaluated on the held-out 20% test set that the model has never seen.

Inference

At inference time, the trained model is loaded from a .joblib file stored in GCS. The app fetches the latest market data for the requested ticker, runs it through the same preprocessing pipeline, and calls model.predict() to get the next-day direction. Response time target is under 3 seconds end-to-end, including the yfinance API call.

3.3 Cloud Service Requirements

Service	Purpose	Spec
Google Cloud Storage (GCS)	Store raw data (CSV), processed features, trained model artifacts (.joblib), and evaluation reports	~500 MB initially, scales with number of tickers
Compute Engine or Cloud Run	Host the Streamlit dashboard and serve predictions	e2-small instance (2 vCPU, 2 GB RAM) or Cloud Run with 1 vCPU / 512 MB
Cloud Build	CI/CD pipeline - auto-build Docker image and deploy on push to main	Triggered on GitHub webhook
Container Registry (GCR)	Store Docker images for the application	One image per build, ~200 MB each
Cloud Logging	Capture application logs, errors, and prediction request metrics	Default retention (30 days)

4. Non-Functional Requirements

4.1 Performance

The dashboard should load in under 5 seconds on a standard broadband connection. Individual predictions (user picks a ticker, hits predict) should return in under 3 seconds, including the time it takes to fetch the latest data from Yahoo Finance. Model training for a single ticker's dataset (5+ years of daily data, roughly 1,200-1,500 rows) should complete in under 60 seconds on the target compute instance.

4.2 Reliability

The system should have 95%+ uptime during the academic term. If the Yahoo Finance API is down or rate-limited, the app should gracefully fall back to the most recently cached data and show a clear message to the user. Model artifacts are versioned in GCS, so we can always roll back to a previous model if a new one underperforms.

4.3 Scalability

For this project, we're targeting a handful of concurrent users (< 10). But the architecture is set up so that if we needed to scale, we could move to Cloud Run (which auto-scales) and add a caching layer for frequently requested tickers. The data pipeline can handle any number of tickers - we just add them to the config list.

4.4 Security

No sensitive user data is collected or stored - the app doesn't have user accounts or personal information. GCP resources are locked down with IAM roles (principle of least privilege). The Streamlit app runs behind HTTPS. API keys and service account credentials are stored as environment variables or GCP Secret Manager, never hardcoded.

4.5 Maintainability

Code is organized into clear modules (data fetching, preprocessing, training, evaluation, serving). Each module has its own directory and README. We use Git with feature branches and pull requests. Dependencies are pinned in requirements.txt. Docker ensures consistent environments across dev and production.

5. Data Requirements

5.1 Data Sources

All data comes from Yahoo Finance through the yfinance library. This is a free, publicly available API that provides daily OHLCV data for thousands of tickers. It's well-documented, widely used in academic and research settings, and sufficient for what we need. No paid data subscriptions are required.

5.2 Data Format and Fields

Field	Type	Description	Unit
Date	datetime	Trading day (index)	YYYY-MM-DD
Open	float64	Opening price for the day	USD
High	float64	Highest price during the day	USD
Low	float64	Lowest price during the day	USD
Close	float64	Closing price (adjusted)	USD
Volume	int64	Number of shares traded	Count

5.3 Dataset Size and Storage

For a single ticker with 5 years of daily data, we're looking at roughly 1,250 rows. If we train on 10-20 tickers, that's 12,500-25,000 rows total - small enough to fit in memory easily. Raw CSV files for 20 tickers run about 5-10 MB. Processed feature files (with all the technical indicators added) roughly double that. Trained model artifacts (.joblib files) are typically 1-5 MB each. Total storage: well under 100 MB for the full project.

5.4 Data Privacy and Retention

All data used is publicly available market data: no PII, no proprietary datasets, no licensing restrictions for academic use. Yahoo Finance's terms of use allow personal and educational use of their data. We retain raw and processed data in GCS for the duration of the project. No data retention policies beyond the academic term are needed.

> *Note: yfinance is not affiliated with or endorsed by Yahoo, Inc. It's an open-source tool that uses Yahoo's publicly available APIs. Refer to Yahoo's terms of use for details on data usage rights.*

6. System Architecture

6.1 Detailed Architecture

The system follows a layered architecture where each component handles one job and passes its output to the next layer. Here's the detailed breakdown.

Data Ingestion Layer

A Python script (data_fetcher.py) uses yfinance to download OHLCV data for configured tickers. It supports both batch mode (download all tickers at once during scheduled runs) and on-demand mode (fetch latest data for a specific ticker when a user requests a prediction). Raw data lands in a GCS bucket under /raw/{ticker}/{date}.csv.

Preprocessing Layer

The feature_pipeline.py module reads raw data, handles missing values, and computes all technical indicators. It outputs a feature matrix where each row is a trading day and columns are the input features plus the target label. Processed data goes to GCS under /processed/{ticker}/features.csv. The pipeline is idempotent; running it twice on the same data produces the same output.

Model Training Layer

train.py loads processed feature files, splits them chronologically, trains both Logistic Regression and Random Forest models, runs hyperparameter tuning via GridSearchCV with TimeSeriesSplit, and saves the best model as a .joblib artifact to GCS under /models/{ticker}/{model_name}_{timestamp}.joblib. A metadata JSON file is saved alongside with training params, metrics, and the feature list used.

Evaluation Layer

evaluate.py loads a trained model and the held-out test set, generates predictions, and computes metrics (accuracy, precision, recall, F1, confusion matrix). Results are saved as JSON and also rendered as plots (confusion matrix heatmap, actual vs. predicted trend chart) stored in GCS under /reports/.

Serving Layer

The Streamlit app (app.py) is the main user-facing component. On startup, it loads the latest trained model from GCS. When a user selects a ticker and clicks "Predict," the app fetches the latest data via yfinance, runs the preprocessing pipeline on it, and calls the model to get a prediction. The whole thing is containerized in a Docker image for consistent deployment.

CI/CD Layer

A cloudbuild.yaml config in the repo root triggers Cloud Build on every push to main. The pipeline builds a Docker image, pushes it to GCR, and deploys it to the target compute instance (or Cloud Run service). If the build fails, the previous version stays live: no downtime.

6.2 Component Communication

Connection	Method	Trigger
Data Fetcher → GCS	Raw CSV upload via GCS Python client	Triggered on schedule or on-demand
GCS → Feature Pipeline	Read CSV from bucket	Pull model
Feature Pipeline → GCS	Processed CSV upload	After preprocessing completes
GCS → Training Module	Read feature CSVs	On training trigger
Training Module → GCS	Model artifact + metadata upload	After training completes
GCS → Streamlit App	Load model artifact on startup	App initialization
yfinance → Streamlit App	Live data fetch for prediction	On user request
GitHub → Cloud Build	Webhook on push to main	CI/CD trigger
Cloud Build → GCR	Docker image push	Build step
Cloud Build → Compute/Run	Deploy new container	Deploy step

7. Training and Evaluation

7.1 Training Pipeline

The training pipeline follows these steps, in order:

1. **Data Ingestion**: Pull 5+ years of daily OHLCV data for each target ticker using yfinance. Save raw CSVs to GCS.
2. **Preprocessing**: Clean data (handle nulls, remove weekends/holidays already excluded by market data). Forward-fill minor gaps. Drop rows with critical missing values.
3. **Feature Engineering**: Compute technical indicators: SMA (5, 10, 20), EMA (12, 26), RSI (14), MACD (12/26/9), Bollinger Bands (20, 2σ), daily returns (pct change), 10-day rolling volatility. Generate binary target label (1 = up, 0 = down).
4. **Train/Test Split**: Chronological 80/20 split. No shuffling. The test set is always the most recent 20% of data, simulating real-world forward testing.
5. **Model Training**: Train Logistic Regression (tuning C, penalty) and Random Forest (tuning n_estimators, max_depth, min_samples_split) using GridSearchCV with 5-fold TimeSeriesSplit on the training set.

6. **Model Selection**: Pick the model+hyperparameter combo with the best cross-validation F1 score. Save as .joblib artifact.
7. **Evaluation**: Run the selected model on the held-out test set. Log all metrics and generate visualizations.

7.2 Evaluation Metrics

We're tracking these metrics to assess model quality:

Metric	What It Tells Us	Target
Accuracy	Overall percentage of correct predictions (up/down)	60-65%
Precision	Of all predicted "up" days, how many were actually up	≥ 58%
Recall	Of all actual "up" days, how many did we correctly predict	≥ 58%
F1 Score	Harmonic mean of precision and recall — balances both	≥ 58%
Confusion Matrix	Breakdown of true positives, true negatives, false positives, false negatives	Visual check
ROC-AUC	Area under the ROC curve — measures discrimination ability	≥ 0.55

A 60-65% accuracy target might sound low, but in financial prediction, consistently beating 50% (random) on out-of-sample data is meaningful. We're being realistic about expectations here. The confusion matrix is especially important; it shows us if the model is biased toward predicting one direction over the other.

8. Deployment Requirements

8.1 Cloud Environment Setup

The production environment runs on Google Cloud Platform. Here's the setup:

- **Project:** A dedicated GCP project with billing enabled (using free tier credits where possible).
- **Region:** us-central1 (low latency, good pricing).
- **Compute:** Cloud Run for the Streamlit app (auto-scales to zero when not in use, saves cost). Alternatively, a single e2-small Compute Engine instance if we need a persistent state.
- **Storage:** One GCS bucket with folders for /raw, /processed, /models, and /reports.

- **Networking:** HTTPS only. No custom VPC needed for this project scope.

8.2 Deployment Strategy

We're going with a straightforward containerized deployment. The Streamlit app and all its dependencies are packaged in a Docker image. This image is built automatically by Cloud Build and deployed to Cloud Run. There's no batch vs. real-time distinction for serving; the app handles predictions on-demand when users interact with the dashboard. Model retraining is a separate, manually triggered process (or could be scheduled weekly via Cloud Scheduler if we want to automate it).

8.3 CI/CD Pipeline

Step	Description
1. Push to Main	Developer pushes code to the main branch on GitHub
2. Webhook Trigger	GitHub webhook notifies Cloud Build
3. Build	Cloud Build pulls the repo, runs tests (pytest), and builds the Docker image
4. Push Image	Docker image is pushed to Google Container Registry
5. Deploy	Cloud Build deploys the new image to Cloud Run (or restarts the Compute Engine container)
6. Health Check	Cloud Run verifies the new revision is healthy before routing traffic to it

8.4 Monitoring and Maintenance

Cloud Logging captures all application logs: errors, warnings, prediction requests, and response times. We set up basic alerts in Cloud Monitoring: if the app returns 5xx errors for more than 5 minutes, or if prediction latency exceeds 10 seconds, we get notified. Model performance is tracked by logging prediction confidence scores and periodically comparing predicted vs. actual outcomes to detect model drift.

9. User Interface Requirements

9.1 Dashboard Requirements

The Streamlit dashboard is the only user-facing component. It needs to be clean, fast, and easy to understand; no financial expertise required to use it. Here's what it includes.

Main Page

- **Ticker Selection:** A dropdown or text input where the user enters a stock ticker (e.g., AAPL, MSFT, TSLA). Validates the ticker against yfinance before proceeding.
- **Predict Button:** Triggers the prediction pipeline. Shows a spinner while processing.
- **Prediction Result:** Clearly displays "UP" or "DOWN" with a confidence percentage. Color-coded (green for up, red for down).
- **Historical Price Chart:** A line chart of the stock's closing price over the last 6 months, with the predicted direction shown as an arrow or annotation on the most recent data point.
- **Technical Indicators Panel:** Toggle to show/hide SMA, EMA, and Bollinger Bands overlaid on the price chart.

Model Performance Page

- **Accuracy Summary:** Table showing accuracy, precision, recall, and F1 for each model.
- **Confusion Matrix:** Heatmap visualization.
- **Actual vs. Predicted Chart:** Shows the model's predictions against actual outcomes over the test period.

9.2 API Documentation (For Developers)

If we expose a REST API endpoint (optional, beyond the Streamlit UI), it follows this spec:

Endpoint	Description	Example Response
GET /predict?ticker=AAPL	Returns next-day prediction for the given ticker	{"ticker": "AAPL", "prediction": "UP", "confidence": 0.63, "model": "RandomForest", "timestamp": "2026-02-18T15:00:00Z"}
GET /health	Health check endpoint	{"status": "ok", "model_loaded": true}
GET /metrics?ticker=AAPL	Returns model evaluation metrics for the ticker	{"accuracy": 0.62, "f1": 0.60, "precision": 0.61, "recall": 0.59}

10. Cost Breakdown and Resource Allocation

10.1 Software and Tools (All Free)

Every tool in this project is open-source or free-tier. No licensing fees, no subscriptions.

Tool	Purpose	Cost	License
Python 3.10+	Primary language	$0	Open-source
Pandas / NumPy	Data manipulation	$0	Open-source
Scikit-learn	ML models and evaluation	$0	Open-source
Matplotlib / Plotly	Data visualization	$0	Open-source
yfinance	Yahoo Finance data API	$0	Open-source
Streamlit	Dashboard framework	$0	Open-source (Community tier)
Docker	Containerization	$0	Docker Desktop free for education
Git / GitHub	Version control and collaboration	$0	Free for public/educational repos

Software Total: $0

10.2 Cloud Infrastructure (Estimated Monthly)

GCP costs are minimal. We use free-tier credits where available and stay on the smallest instances.

Service	Usage Estimate	Pricing	Est. Monthly Cost	Notes
GCS Standard Storage	~100 MB for data + models	$0.02/GB/month	$0.01	Free tier covers 5 GB

Cloud Run	Streamlit app hosting	$0 (free tier: 2M requests/month)	$0.00	Auto-scales to zero when idle
Cloud Build	CI/CD pipeline	$0 (free tier: 120 min/day)	$0.00	Well within free limits
Container Registry	Docker image storage	$0.02/GB/month	$0.01	~200 MB per image
Cloud Logging	Application logs	$0 (free tier: 50 GB/month)	$0.00	Minimal log volume
Compute Engine (backup)	e2-small if Cloud Run not used	$13.80/month	$13.80	Only if needed

Cloud Total (Cloud Run path): ~$0.02/month; essentially free within GCP free tier. **Cloud Total (Compute Engine path): ~$13.82/month**; only if we need a persistent VM.

10.3 Data Costs

Source	Data Type	Cost	Notes
Yahoo Finance (via yfinance)	Historical OHLCV data	$0	Free for academic use

Data Total: $0

10.4 Human Resources

Team Member	Primary Responsibilities	Estimated Weekly Hours
Nathan Kennedy	Model development, training pipeline, evaluation	~8-10 hrs/week
Batyr Rasulov	Data pipeline, cloud infrastructure, CI/CD, deployment	~8-10 hrs/week
Alejandro Barranco-Leyte	Frontend dashboard, visualization, testing, documentation	~8-10 hrs/week

Labor Cost: $0 (student project no paid labor).

10.5 Total Project Cost Summary

Category	Cost
Software & Tools	$0.00
Cloud Infrastructure (Cloud Run)	$0.02/month
Data Acquisition	$0.00
Human Resources	$0.00 (student project)
Total Estimated Project Cost	**~$0.02/month**

The big takeaway here is that this project runs on almost nothing financially. Open-source tools, free-tier cloud services, and publicly available data mean we can focus entirely on building a good system without worrying about budget.

11. Risk Assessment

Every project has risks. Here's what we've identified and how we plan to handle each one.

Risk	Likelihood	Impact	What Happens	How We Handle It
Market Noise / Randomness	High	Medium	Reduced prediction accuracy	Focus on trend classification, not exact prices. Set realistic accuracy targets (60-65%). Use multiple technical indicators to capture different signal types.
Model Overfitting	Medium	High	Good training results but poor real-world performance	Use TimeSeriesSplit (not random CV). Limit model complexity (max_depth, regularization). Always evaluate on held-out test data that the model hasn't seen.
Yahoo Finance API Downtime	Low	Medium	Can't fetch data for predictions	Cache the most recent data locally. Show clear error messages. The trained model still works;e we just can't get new data temporarily.

Data Quality Issues	Medium	Medium	Biased or incorrect predictions	Validate data on ingestion (check for nulls, outliers, stale data). Log data quality metrics. Drop tickers with > 2% missing data.
Scope Creep	Medium	Medium	Project not finished on time	Stick to the defined scope. No new features without removing something else. Weekly check-ins to stay on track.
Cloud Cost Overrun	Low	Low	Unexpected charges	Stay within the free tier. Set budget alerts at $5 and $10. Use Cloud Run (scales to zero) instead of always-on VMs.

The biggest risk is fundamentally inherent to the problem; stock markets are noisy and hard to predict. That's why we framed this as trend classification with modest accuracy targets, not price prediction. Everything else is manageable with standard engineering practices.